

Spring MVC - REST



P.Mathieu

IUT de Lille
<http://www.iut-a.univ-lille.fr>
prenom.nom@univ-lille.fr

Plan



REST en Spring

REST en Spring

REST : Representationnal State Transfer



Aujourd'hui trois grands types d'architectures "web" cohabitent :

- ❶ **avec Vue** (Web historique, Modèle MVC).
 - ▶ Le rendu (View) est fabriqué côté serveur
 - ▶ L'objectif est de fournir du HTML
 - ▶ Usage uniquement des méthodes `Get` et `Post`
- ❷ **Micro-services.**
 - ▶ L'objectif est uniquement de renvoyer des données (JSON, XML, YAML)
 - ▶ Usage de la totalité des ordres HTTP
 - ▶ Séparation forte "Back" et "Front"
 - ▶ Plusieurs formats historiques (sur XML) : WSDL, SOAP, UDDI,...
- ❸ **S.P.A.** Avec un serveur React ou Angular qui pilote les appels Back

REST en Spring



Qu'est-ce qu'une API REST ?

Style architectural qui permet aux logiciels de communiquer entre eux sur un réseau.

une API REST expose des ressources avec lesquelles il est possible d'interagir via le protocole HTTP

Idéalement, le client peut demander des ressources avec un langage que le serveur comprend, et le serveur renvoie la ressource avec un langage que le client accepte.

REST en Spring

Les règles de base

- ▶ Règle n°1 : l'URI comme identifiant des ressources.
`http://localhost:8080/etudiants/5`
- ▶ Règle n°2 : les verbes HTTP comme identifiant des opérations.
CRUD $\equiv$ Post, Get, Put/Patch, Delete
- ▶ Règle n°3 : les réponses HTTP comme représentation des ressources.
selon l'entête Accept : HTML, XML, JSON, YAML ...
- ▶ Règle n°4 : les liens comme relation entre ressources.
- ▶ Règle n°5 : Pas d'état entre deux requêtes (stateless). Par ex un paramètre comme jeton d'authentification.

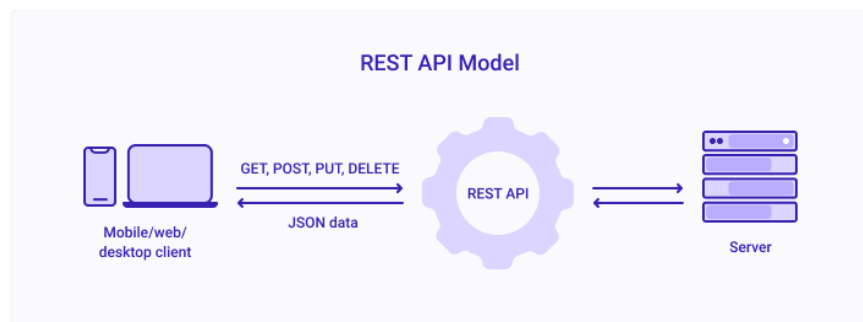
REST en Spring

Historique

- ▶ JAX-RS (Java API for RESTful Web Services) est une norme de description d'une architecture REST.
- ▶ Jersey est l'implémentation de référence de JAX-RS développée par Oracle (fondation Eclipse)
- ▶ JAX-RS s'est inspiré de Spring-MVC qui permettait déjà du REST
- ▶ Spring-MVC n'implémente pas vraiment JAX-RS
- ▶ Spring est beaucoup plus puissant et plus utilisé que Jersey (SpringMVC ne fait pas que du REST, et il en plus il le fait mieux)

REST en Spring

En résumé ...



©<https://stallionemma88.medium.com/what-is-a-rest-api-a8a382975378>

REST en Spring

Représentation de la ressource

- ▶ la représentation souhaitée peut avoir plusieurs formes : JSON, XML, YAML ...
- ▶ le format utilisé est identifié par un type MIME
- ▶ le client précise le format souhaité grâce au champ d'entête Accept
- ▶ le serveur précise le format envoyé avec le champ d'entête Content-Type

Format application/json

```
{ "id":5,
  "prenom":"Emma",
  "nom": "Bovary",
  "age": 24,
  "groupe": "A"
}
```

Format application/xml

```
<Etudiant>
  <id>5</id>
  <prenom>Emma</prenom>
  <nom>Bovary</nom>
  <age>24</age>
  <groupe>A</groupe>
</Etudiant>
```

REST en Spring

Rappel : requête HTTP

Une requête HTTP est constituée de 4 éléments :

- 1 Une ligne de requête (méthode, chemin, version HTTP)
- 2 Une série d'en-têtes HTTP, (CRLF après chacun)
- 3 Une ligne vide
- 4 Un corps de requête, si nécessaire. (aucun pour GET par exemple)

```
POST /users HTTP/1.1
Accept: application/json, */*;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Content-Length: 65
Content-Type: application/json
Host: jsonplaceholder.typicode.com
User-Agent: HTTPie/3.2.1

{
  "email": "jb@is.gb",
  "name": "James Bond",
  "phone": "+(33) 007"
}
```

REST en Spring

Les contrôleurs

Spring offre deux types de contrôleurs

- **@Controller**
Contrôleur classique avec des méthodes qui renvoient vers des vues
- **@RestController**
qui est équivalent à
 @Controller
 @ResponseBody

On crée en général 1 contrôleur par entité, chacun avec les méthodes HTTP à traiter

REST en Spring

Les contrôleurs

Chaque méthode du contrôleur est annotée par le verbe HTTP correspondant

- @PostMapping ... créer une ressource
- @GetMapping ... récupérer une ou des ressources
- @PutMapping ... modifier une ressource
- @PatchMapping ... modifier partiellement une ressource
- @DeleteMapping ... détruire une ressource

REST en Spring

Du bon usage de l'URL

On utilise uniquement le nom de la ressource dans l'URI, car l'action se trouve déjà dans le verbe HTTP !

INCORRECT

POST /createClient

PUT /updateClient

GET /getClient

CORRECT

POST /clients

PUT /client/{id}

GET /client/{id}

REST impose de mettre les identifiants dans le chemin de l'URL

REST en Spring

Binding des "path variables"

Spring permet de récupérer automatiquement les variables de l'url

```
@RestController
@RequestMapping("/personne")
public class PersonneController {
    @GetMapping("/{id}")
    public Personne recup(@PathVariable("id") int id)
    { Personne p = personneRepository.findById(id);
      return p;
    }
}
```

- ▶ Bien distinguer `PathVariable` et `RequestParam`. Ces deux annotations ne sont pas incompatibles
- ▶ Il est possible de tout récupérer d'un coup en mettant une Map comme argument :

```
@PathVariable Map<String,String> pathvars
```

REST en Spring

Requetes REST "standard"

Partons d'une classe `Produits`

Le contrôleur correspondant contiendra au minimum les endpoints suivants :

- ▶ Requête POST sur `/Produits` : ajoute un nouveau produit ;
- ▶ Requête GET sur `/Produits` : affiche la liste de tous les produits ;
- ▶ Requête GET sur `/Produits/{id}` : affiche un produit spécifique ;
- ▶ Requête PUT sur `/Produits/{id}` : met à jour un produit ;
- ▶ Requête DELETE sur `/Produits/{id}` : supprime un produit.

REST en Spring

Accès aux données du corps de requête et de réponse

- ▶ Avec `@RequestBody` toute donnée du corps est sérialisée en objet Java
- ▶ Avec `@ResponseBody`, tout objet renvoyé est sérialisé dans le corps de la réponse (JSON par défaut, via Jackson)

```
@RestController
public class PersonneController {
    @PostMapping("/personnes")
    public Client insert(@RequestBody Client client)
    {
        repository.save(client);
        return client;
    }
}
```

Une appli RESTFUL en Spring-JPA devient quasi immédiate !

REST en Spring

Un contrôleur REST classique

```
@RestController
@RequestMapping(value = "/clients")
public class ClientController {
    @Autowired
    private ClientRepository repository;

    @GetMapping(value={"/", "/"})
    public List<Client> selectAll(){
        List<Client> clientList = repository.findAll();
        return clientList;
    }

    @GetMapping("/{id}")
    public Client getSpecificClient(@PathVariable int id){
        return repository.findById(id);
    }

    @PostMapping("")
    public void insert(@RequestBody Client client) {
        repository.save(client);
    }

    @PatchMapping("/{id}")
    public void update(@PathVariable int id, @RequestBody Client client) {
        Client oldClient = repository.findById(id);
        ...
        repository.save(oldClient);
    }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable int id) {
        Client deleteClient = repository.findById(id);
        repository.delete(deleteClient);
    }
}
```

REST en Spring

Le Binding en résumé

Plusieurs annotations, utilisées dans la signature de méthode, permettent de lier les données des requêtes avec des variables et objets Java :

- ▶ `@RequestParam` paramètre de l'url
- ▶ `@PathVariable` paramètre du chemin d'url
- ▶ `@RequestHeader` paramètre du header
- ▶ `@RequestBody` objet du corps de req
- ▶ `@ModelAttribute` objet issu du modèle

REST en Spring

Les outils de requêtage

- ▶ Un navigateur classique ... mais ne traite que GET (et indirectement POST)
- ▶ **cURL**
 - ▶ -X, la méthode HTTP à utiliser
 - ▶ -d, les données à envoyer
 - ▶ -H, headers à envoyer (Content-Type, Accept, Authorization, ...)
 - ▶ -i, affiche les headers en plus de la réponse
 - ▶ -l, affiche uniquement les headers
- ▶ **httpie** (un curl++ : nettement plus facile)
- ▶ **RESTED** ou **RestClient** (extension Firefox, Chrome, Safari)
- ▶ **Vscode {REST Client}** extension
- ▶ **Postman** (compte obligatoire) , **Bruno**
- ▶ ...

REST en Spring

Interrogation avec Curl

```
curl -X GET localhost:8080/clients/

curl -X GET localhost:8080/clients/5

curl -X POST localhost:8080/clients -H 'Content-Type: application/json'
-d '{"nom":"duchemin","prenom":"paul","age":"50","groupe":"A"}'

curl -X PUT localhost:8080/clients/11 -H 'Content-Type: application/json'
-d '{"nom":"dubois","prenom":"paul","age":"50","groupe":"A"}'

curl -X DELETE localhost:8080/clients/11
```

REST en Spring

Codes d'erreurs HTTP

En REST, le code d'erreur HTTP devient capital ! (liste complète ici)

- 200** (ok)
- 201** (created) uniquement pour POST
- 400** (bad request) paramètres erronés
- 404** (not found)
- 405** (not allowed) un delete sur collection par ex
- 409** (conflict) un post sur une ressource existante par ex
- 415** (Media Type) type MIME non pris en charge
- 500** (Internal server error) une exception, erreur générique du fainéant

REST en Spring

La gestion des codes d'erreurs



- ▶ On rappelle qu'une réponse HTTP est en 3 parties : header, body et status
- ▶ Renvoyer les data c'est bien, renvoyer les code HTTP avec, c'est mieux !
- ▶ Depuis Spring 6, et SpringBoot 2.7, Spring introduit la classe `ResponseEntity` qui encapsule header, body et status.
- ▶ En un seul objet, on peut renvoyer deux de ces informations, voire les 3 !
- ▶ Equivalent à :

```
response.setHeader(header); // n fois
response.setStatus(code);
response.getWriter().print(obj);
```

REST en Spring

La gestion des codes d'erreurs (I)



Gestion des erreurs méthode par méthode : l'object `ResponseEntity`

```
@GetMapping("/hello")
ResponseEntity<String> hello() {
    return new ResponseEntity<>("Hello World!", HttpStatus.OK);
}

@GetMapping(value =("/{id}")
public ResponseEntity<Client> getById(@PathVariable("id") Integer id) {
    Optional<Client> client = clientRepo.findById(id);
    if(client.isPresent())
        return new ResponseEntity<>(client.get(), HttpStatus.OK);
    else return new ResponseEntity<>(HttpStatus.NOT_FOUND);
}
```

REST en Spring

La gestion des codes d'erreurs (II)



“factorisation” du traitement des erreurs : `@ExceptionHandler`

- ▶ L'objet `ProblemDetail` permet de sérialiser en JSON le code et le message d'erreur
- ▶ L'annotation `@ExceptionHandler` permet de lier cet objet à un type d'exception

```
@ExceptionHandler({ConstraintViolationException.class})
public ProblemDetail details(ConstraintViolationException erreur)
{
    ProblemDetail problem =
        ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST,
            erreur.getMessage());

    return problem;
}
```

Il est bien sûr possible d'utiliser cela avec ses propres exceptions

REST en Spring

Input/Output du body dans différents formats



- ▶ Une bonne appli REST fournit le format que le client souhaite !
- ▶ Spring/Jackson savent gérer de nombreux formats (JSON par défaut)

```
@RequestMapping(value="/etudiants",
    produces = {MediaType.APPLICATION_XML_VALUE,
        MediaType.APPLICATION_JSON_VALUE })

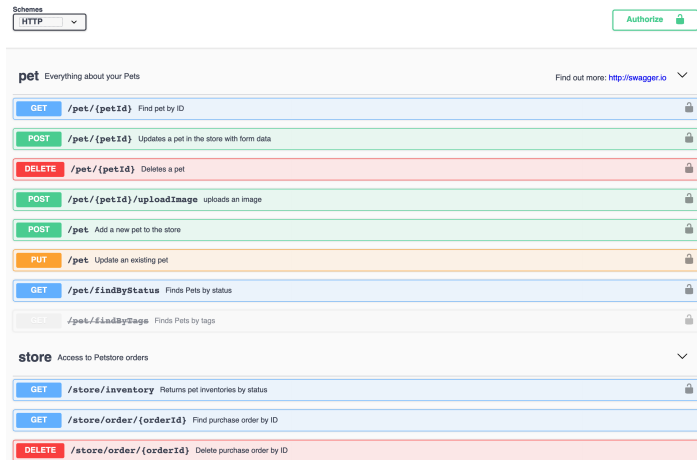
<dependency>
<groupId>com.fasterxml.jackson.dataformat</groupId>
<artifactId>jackson-dataformat-xml</artifactId>
<version>2.20.1</version>
</dependency>
```

```
curl -i -X GET -H "Accept: application/json" localhost:8080/etudiants/
curl -i -X GET -H "Accept: application/xml" localhost:8080/etudiants/
```

-i pour avoir l'entete avec le code d'erreur

REST en Spring

OPENAPI (ex Swagger)



REST en Spring

OPENAPI

Springdoc Open API

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>3.0.0</version>
</dependency>
```

Documentation du projet accessible immédiatement sur :

<http://localhost:8080/swagger-ui.html>

(html)

<http://localhost:8080/v3/api-docs>

(json)

Voir les **SpringDoc** annotations et la **spécification openapi**. Voir aussi l'outil **redoc**.